

Tu propio repositorio en GitHub

Manual de la sesión — crearás tu repositorio, guardarás tus primeros cambios con `git add` y `git commit`, y los publicarás con `git push`.

Modalidad: práctica guiada en clase **Fecha:** miércoles, 15 de julio de 2026

Trae: tu computadora con todo lo de la sección «Antes de la clase»

Antes de la clase — requisitos, no tarea

Esta sesión continúa exactamente donde terminó la anterior: hoy *tú* produces y publicas. **No repetiremos los pasos de instalación ni los de la clase 2;** verifica esta lista y, si te falta algo, resuélvelo **antes** de la clase por el canal de ayuda.

⚠ Verifica que ya tienes:

- **Las tres herramientas funcionando**

— VS Code, Python y Git, con tu identidad configurada en Git (tu nombre y tu correo). Todo está en el

Manual de instalación

.

- **La práctica de la clase 2 completa**

— el repositorio del curso clonado en `Documentos/propedeutico` y tu entorno `.venv` creado. Si faltaste o algo quedó a medias, sigue el

Manual de la clase 2

.

- **Tu cuenta de GitHub con el correo verificado**

— hoy publicarás en ella. Comprueba que puedes iniciar sesión en github.com/login.

Tres ideas antes de tocar el teclado

Commit

Una fotografía de tu proyecto en un momento dado: qué archivos había, qué decían, quién tomó la foto y cuándo, con un mensaje que explica el cambio. El historial de un repositorio es un álbum de commits.

Preparación (add)

Antes de la foto, decides quién sale en ella: `git add` coloca tus cambios en la «bandeja de preparación». Solo lo que está en la bandeja entra en el próximo commit.

Publicar (push)

Tus commits nacen en tu computadora. `git push` los sube a GitHub, donde quedan respaldados y visibles. Es el gesto inverso al `git pull` con que bajas los materiales del curso.

EN LA CLASE • LO HAREMOS JUNTOS

Práctica guiada: crear, guardar y publicar

1

Crea tu repositorio en GitHub

1. Entra a github.com con tu cuenta y pulsa el botón `+` (arriba a la derecha) → `New repository`.
2. En `Repository name` escribe exactamente `mi-propedeutico` — minúsculas, sin espacios, tildes ni ñes: la regla de oro de la clase 2 también aplica aquí.
3. En `Description` (opcional) puedes poner: `Bitácora del propedéutico – FLACSO Ecuador`.
4. Deja la visibilidad en `Public`: este repositorio es el inicio de tu portafolio.
5. Marca la casilla `Add a README file` — así el repositorio nace con un primer archivo (y un primer commit hecho por GitHub).
6. Pulsa `Create repository`. GitHub te lleva a la página de tu repositorio nuevo.

Fíjate en la dirección del navegador: `github.com/tu-usuario/mi-propedeutico`. Ese es **tu** espacio público; el del curso era el del docente.

2

Clónalo junto al repositorio del curso

1. En la página de tu repositorio, pulsa el botón verde `<> Code` → pestaña `HTTPS` → icono de copiar (igual que en la clase 2).
2. Abre VS Code y ve a `File → Open Folder` (Archivo → Abrir carpeta), selecciona la carpeta `propedeutico` de tus Documentos (la carpeta madre, **no** la del repositorio del curso).
3. Abre una terminal (`Terminal → New Terminal`) y verifica que la ruta antes del cursor termina en `propedeutico`.

4. Escribe `git clone` , un espacio, y pega la dirección que copiaste en el punto 1 (clic derecho → Pegar, o `Ctrl+V` / `⌘V`). Presiona Enter. Quedará así, pero con **tu** nombre de usuario donde el ejemplo dice `tu-usuario` :

```
git clone https://github.com/tu-usuario/mi-propedeutico.git
```

Al terminar, `propedeutico` contiene dos carpetas hermanas: la del curso y la tuya:

TU CARPETA DE TRABAJO, A PARTIR DE HOY

Documentos/propedeutico/

├─ propedeutico-flacso/ ← el del curso: solo lees y haces git pull

└─ mi-propedeutico/ ← el tuyo: aquí escribes, haces commit y push

⚠ Un repositorio nunca va dentro de otro

Clona siempre desde la carpeta madre `propedeutico`. Si la terminal estaba «parada» dentro de `propedeutico-flacso` al clonar, tu repositorio quedó anidado dentro del repositorio del curso — revisa P6 para moverlo.

Ahora abre **tu** repositorio como carpeta de trabajo: `File → Open Folder` → `mi-propedeutico` → confía en los autores (eres tú).

3

Crea tu primer archivo: la bitácora del curso

1. En el explorador de VS Code (barra izquierda), haz clic derecho en el área vacía bajo la lista de archivos → **New File** (Nuevo archivo) y nómbralo `bitacora.md` — el nombre sin tildes; el *contenido* puede llevarlas con total normalidad. Debe quedar al mismo nivel que `README.md`.
2. Escribe algo como esto (o tu propia versión) y **guarda** con `Ctrl+S` / `⌘S`:

```
# Bitácora del propedéutico

## Clase 3 – 15 de julio de 2026

Hoy creé mi primer repositorio y aprendí el ciclo
editar → add → commit → push.
```

Los archivos `.md` (Markdown) son texto plano con formato sencillo: `#` marca un título, `##` un subtítulo. GitHub los muestra ya formateados — por eso el README de cualquier repositorio se ve «bonito».

4

git status — pregunta a Git qué ve

Abre una terminal nueva (`Terminal → New Terminal`): como tu carpeta de trabajo ahora es tu repositorio, la ruta antes del cursor termina en `mi-propedeutico`. Ahí escribe el comando. `git status` no cambia nada: solo informa — úsalo todo el tiempo, antes y después de cada paso:

```
git status
```

RESULTADO ESPERADO

```
~/Documentos/propedeutico/mi-propedeutico> git status
```

```
On branch main
```

```
Your branch is up to date with 'origin/main'.
```

```
Untracked files:
```

```
(use "git add <file>..." to include in what will be committed)
```

```
  bitacora.md
```

```
nothing added to commit but untracked files present (use "git add" to track)
```

Cómo leerlo: *untracked* (en rojo) significa que Git ve el archivo nuevo pero todavía no lo vigila ni lo incluirá en ninguna fotografía. La propia salida te sopla el siguiente paso: `git add`.

5

git add — prepara la fotografía

Coloca el archivo en la bandeja de preparación:

```
git add bitacora.md
```

No imprime nada si todo va bien. Comprueba con `git status`: el archivo ahora aparece en verde, listo para el commit:

RESULTADO ESPERADO

```
~/Documentos/propedeutico/mi-propedeutico> git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   bitacora.md
```

💡 Más adelante usarás `git add .` (con punto) para preparar *todos* los cambios pendientes de una vez.

6

git commit — toma la fotografía

El commit guarda lo preparado en el historial, con un mensaje entre comillas que explica el cambio:

```
git commit -m "Crea la bitacora del curso"
```

RESULTADO ESPERADO (EL CÓDIGO VARÍA)

```
~/Documentos/propedeutico/mi-propedeutico> git commit -m "Crea la bitacora
[main a1b2c3d] Crea la bitacora del curso
 1 file changed, 6 insertions(+)
 create mode 100644 bitacora.md
```

💡 Mensajes de commit que sirven

Escribe qué hace el cambio, en presente y de forma concreta: «Crea la bitacora del curso», «Añade la entrada de la clase 4». Dentro de unas semanas, `git log` será tu memoria — un historial de mensajes tipo «cambios» o «avance» no le dice nada a nadie, ni a ti. ¿Y por qué «Añade», sin tilde ni eñe? Git las acepta sin

problema, pero muchos proyectos las evitan en los mensajes porque algunas terminales antiguas las muestran mal; sigue la costumbre que prefieras — en tus *archivos*, escribe siempre con ortografía normal.

El `-m` es importante: sin él, Git abre un editor para pedirte el mensaje (si te pasó, revisa P5).

7

git push — publica en GitHub

Hasta aquí, el commit existe solo en tu computadora. Súbelo:

```
git push
```

⚠ La primera vez, GitHub te pedirá identificarte

Es normal y ocurre una sola vez — y por eso el push se hace **siempre desde la terminal integrada de VS Code**, no desde la Terminal del sistema. `WIN` Se abrirá una ventana del navegador (la lanza *Git Credential Manager*): inicia sesión con tu cuenta de GitHub y pulsa `Authorize`. `MAC` Primero VS Code muestra un pequeño aviso pidiendo permiso — acéptalo (`Allow`) y después se abre el navegador para autorizar. Tu computadora quedará autorizada para los siguientes push. Si la terminal se queda esperando o falla, revisa P2 y P3.

RESULTADO ESPERADO (LOS NÚMEROS VARÍAN)

```
~/Documentos/propedeutico/mi-propedeutico> git push
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 428 bytes | 428.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (0/0), done.
To https://github.com/tu-usuario/mi-propedeutico.git
   e5f6a7b..a1b2c3d  main -> main
```

Ahora ve al navegador, entra a `github.com/tu-usuario/mi-propedeutico` y actualiza la página: ahí está `bitacora.md`, y el contador dice **2 commits** (el README de GitHub y el tuyo). Acabas de publicar código por primera vez.

8

El ciclo completo, una vez más

Lo que convierte esto en hábito es repetirlo. Edita `bitacora.md` — añade al final una línea nueva, por ejemplo «*Publiqué mi primer repositorio.*» — guarda con `Ctrl+S` / `⌘S`, y recorre el ciclo entero:

```
git status
git add .
git commit -m "Anade la primera entrada de la bitacora"
git push
```

Refresca tu repositorio en el navegador: la página muestra **3 commits**. Pulsa sobre ese contador para ver tu historial — cada commit, con tu nombre, fecha y mensaje.

Este es el ciclo que repetirás toda la maestría: editar → `git add` → `git commit` → `git push`. Tu bitácora crecerá una entrada por clase; ese será tu ejercicio permanente.



Salida de la clase

Antes de cerrar la laptop, verifica que puedes marcar todo:

- ☐ Mi repositorio `mi-propedeutico` existe en mi cuenta de GitHub y es público.
- ☐ Lo cloné junto al repositorio del curso (carpetas hermanas, no anidadas) y lo tengo abierto en VS Code.
- ☐ Hice al menos **dos commits propios** con mensajes descriptivos.
- ☐ Mis cambios se ven en la página de mi repositorio en `github.com`, desde el navegador.
- ☐ Puedo recitar el ciclo sin mirar: editar → `add` → `commit` → `push`.
- ☐ Distingo los dos repositorios: en el del curso solo `git pull`; en el mío, todo lo demás.

El ciclo de Git en seis comandos

Tu tabla de consulta rápida de esta sesión. Los cuatro primeros son el ciclo diario; los dos últimos, tus aliados de consulta:

Comando	Qué hace
<code>git status</code>	Informa qué ve Git: archivos nuevos, modificados o preparados. No cambia nada.
<code>git add archivo</code> · <code>git add .</code>	Prepara un archivo (o todos, con el punto) para el próximo commit.
<code>git commit -m "mensaje"</code>	Guarda la fotografía en el historial, con tu nombre, la fecha y el mensaje.
<code>git push</code>	Sube tus commits a GitHub. Hasta entonces, solo existen en tu computadora.
<code>git log --oneline</code>	Muestra el historial resumido: un commit por línea. Sal con la tecla <code>q</code> si ocupa más de una pantalla.
<code>git pull</code>	Baja las novedades desde GitHub — lo que haces en el repositorio del curso cada semana.

Si algo se atasca

▼ P1 · El commit responde “Please tell me who you are”

Git necesita saber quién toma la fotografía y falta tu identidad (sección G del [Manual de instalación](#)). Configúrala una sola vez, sustituyendo `Nombre Apellido` y `tu-correo@ejemplo.com` por tus datos reales:

```
git config --global user.name "Nombre Apellido"
git config --global user.email "tu-correo@ejemplo.com"
```

Usa el mismo correo de tu cuenta de GitHub y repite el `git commit`.

▼ P2 · Al hacer push se abre el navegador pidiendo autorización (o no pasa nada)

La ventana del navegador es el camino correcto: inicia sesión y pulsa **Authorize**. Si la terminal se queda esperando y no ves nada: **WIN** mira la barra de tareas — a veces la ventana del navegador se abre detrás de VS Code; **MAC** busca el pequeño aviso de VS Code pidiendo permiso y pulsa **Allow** — el navegador se abre después. Ocurre solo la primera vez; después, `git push` pasa directo.

▼ P3 · `git push` falla con “Authentication failed” o pide contraseña en la terminal

GitHub ya no acepta tu contraseña escrita en la terminal. Si te la pidió, cancela con **Ctrl+C** y asegúrate de estar en la **terminal integrada de VS Code** (no en la Terminal del sistema). Repite `git push` y esta vez acepta todo lo que aparezca: el aviso de VS Code pidiendo permiso (**Allow**) y la ventana del navegador (**Authorize**). Si sigue fallando, avisa al docente — se resuelve en un minuto y no es culpa tuya.

▼ P4 · “nothing to commit, working tree clean” — pero yo sí cambié el archivo

Casi siempre el archivo no está **guardado**: mira la pestaña en VS Code — un punto • en lugar de la X indica cambios sin guardar. Guarda con **Ctrl+S** / **⌘S** y repite `git status`. Si ya estaba guardado, puede que estés en la carpeta equivocada: `pwd` debe terminar en `mi-propedeutico`.

▼ P5 · Olvidé el `-m` y la terminal se convirtió en algo extraño

Git abrió un editor para pedirte el mensaje. Dos escenarios:

1. **Pantalla oscura dentro de la terminal (Vim):** presiona **Esc**, escribe `:q!` y presiona Enter. Sales sin guardar y el commit se cancela.
2. **Se abrió una pestaña en VS Code (COMMIT_EDITMSG):** escribe el mensaje en la primera línea, guarda y cierra la pestaña — el commit se completa.

En cualquier caso, la próxima vez: `git commit -m "mensaje"`.

▼ P6 · Cloné mi repositorio dentro del repositorio del curso

Pasa si la terminal estaba «parada» en `propedeutico-flacso` al clonar. Git se confunde con repositorios anidados, así que muévelo: cierra VS Code y, con el explorador de archivos o el Finder, arrastra la carpeta `mi-propedeutico` completa hacia afuera, a `Documentos/propedeutico`, junto a la del curso. No se rompe nada; vuelve a abrirla desde VS Code.

▼ P7 · Hice commit pero en github.com no aparece nada nuevo

El commit vive en tu computadora hasta que lo empujas: ejecuta `git push` y refresca la página. Si push responde `Everything up-to-date`, entonces no hay commits pendientes de subir — revisa `git status`: probablemente el cambio no pasó por `add` y `commit`.

💡 ¿Tu problema no está en la lista? Pídele ayuda a una IA

Un asistente de IA (ChatGPT, Claude, Gemini o Copilot) resuelve muy bien este tipo de atascos **si le das contexto suficiente**. Copia esta plantilla, rellena los huecos [entre corchetes] y pégala en el chat:

```
Estoy siguiendo un manual del curso propedéutico de FLACSO
y me atasqué en [paso o sección del manual donde estabas].
Mi sistema operativo es [Windows o Mac].
Escribí este comando o código:
[pega aquí exactamente lo que escribiste]
Y obtuve este mensaje de error o problema:
[pega aquí el mensaje completo, copiado desde la terminal]
Por favor: primero explícame por qué ocurre el problema,
en lenguaje sencillo (no tengo experiencia en programación),
y después dame los pasos para resolverlo, uno por uno.
```

Tres consejos para que la respuesta sirva:

- **Copia el error completo como texto** — selecciónalo en la terminal y cópialo — mejor que una captura de pantalla o un fragmento suelto.
- **Lee cada comando que la IA te sugiera antes de ejecutarlo**; si no entiendes qué hace, pregúntale. Nunca ejecutes a ciegas comandos que borren archivos o carpetas.
- Si la primera respuesta no funciona, **cuéntale qué pasó al intentarlo**: la conversación de ida y vuelta es parte normal del proceso.

Qué sigue

Ya dominas los dos sentidos del tráfico: *recibir* con `git pull` (clase 2) y *publicar* con `add` → `commit` → `push` (hoy). En la próxima sesión volvemos a los notebooks — algoritmos, tipos de variables y operadores en Python — y tu bitácora empieza a trabajar: al final de cada clase, escribe una entrada nueva y publícala. Ese pequeño ritual es tu respaldo, tu registro de avance y el primer contenido de tu portafolio público.

Propedéutico de Matemáticas y Programación · FLACSO Ecuador — Manual de sesión: tu propio repositorio, versión julio 2026.

Continúa el **Manual de la clase 2** (GitHub y tu primer entorno) y complementa al **Manual de instalación** (VS Code, Python y Git).